Differential Attestation: Accepting Agent-Written Changes from Recordings, Proofs and Declared Relations, Never from the Producer's Account

Assem Chammah
assem@gpt.nexus
Nexus
Brussels, Belgium

ABSTRACT

Coding agents merge more changes than human review can examine: in one repository we operate, 917 changes in 13 days, against about ten a day for a reviewer. The usual substitute, a language model that reads the change, was measured on 144 defects that escaped review in six repositories: the strongest reader identified 10 of 43 design-set defects for the right reason, refused 22 of 43 clean changes, and two readers from different vendors missed the same defects. We present Differential Attestation, an acceptance gate that reads neither the change text nor any statement by its producer. Three instruments supply its evidence. A *recording*: the system runs under a fixed stimulus set before and after the change; any difference outside a scope the requester selected is refused, and the difference inside it must equal an edit the requester made on a real recording. A *proof*: for every function outside the scope, a differential equivalence checker, lean, proves the candidate equal to the parent or returns an input on which they differ. A *relation*: for every artifact that is not a function (build and pipeline configuration, locale files, schemas, documents), a predicate derived from an authoritative artifact or declared outside the change compares the two versions. On the same defects the recording checks caught 22 of 43, 2.2 times the reader, and refused 4 of 43 clean changes; the proof lane raised the count to 31, the artifact lane to 40 of 43. lean caught 15 of 43 design-set defects and 18 of 101 held out, decided 132 of 145 untouched functions equal, found a separating input for 120 of 143 planted mutations and proved none of them equal; its reach is 53 percent of 39,775 functions in 36 repositories. Four bounds fix what a gate can observe and a proof can decide; three independent attacks locate the release step's remaining condition.

KEYWORDS

coding agents, code review, acceptance gates, differential testing, program equivalence, reproducible builds

1 INTRODUCTION

A *coding agent* receives a request in natural language, edits a software repository and proposes the result as a *change*. In one repository we operate, agents merged 917 changes in 13 days; a human reviewer examines about ten a day. Review capacity no longer bounds the defect rate of what is merged.

The common replacement is a language model that reads the change text and reports defects. We measured it on 144 defects that escaped review (Section 4). The strongest reader identified the repaired defect in 10 of 43 design-set cases and refused 22 of 43 clean changes; two readers from different vendors missed the same defects ($\phi = 0.49$), the dependence Knight and Leveson measured between independently written programs [27]. In most cases the escaped defect was not in the text a reader receives: a request a provider rejects, a file omitted from the build, a sibling file not updated, a cache key missing an input, a job moved to a runner too small for it.

This paper presents a *gate*, an acceptance procedure, that does not read the change. Differential Attestation has three properties: no input to its verdict is written by the producer; the verdict is a deterministic function of the system's recordings, of the candidate's source and of relations declared between its artifacts; and its only permitted error is an unneeded refusal. Three instruments supply the evidence. A *recording*: the requester fixes the request as an edit of a real recording of the parent; the gate records the system before and after the change under a fixed stimulus set, refuses every difference outside the request and requires the difference inside it to equal the edit. A *proof*: for every function the request did not name, a differential equivalence checker proves the candidate's function equal to the parent's for every input, or returns an input on which they differ. A *relation*: for every artifact that is not a function, a predicate derived from an authoritative artifact or declared outside the change compares the two versions. A one-use token releases the build that was checked.

We make the following contributions:

- The architecture (Section 3): six checks over recordings, a source lane (lean) with a stated decidable subset of the language, and an artifact lane with a registry of predicates, each verdict carrying its evidence class. The producer's description of its change is never an input.
- Four bounds (Section 3.6) that fix the design: identical recordings receive identical verdicts, rewording a request adds no information about intent, a balance has a null space, and equivalence is decidable only inside a stated subset.
- A corpus of 144 escaped defects with clean twins and 72 pre-registered experiments (Section 4): on the same 43 defects the gate catches 40 of 43 against the reader's 10, and refuses 6 clean changes against 22. Every design that failed is listed with its measured reason (Appendix B).

2 BACKGROUND AND RELATED WORK

2.1 Why reading the change fails and what relates

Reading the change. A reader's input is the change text, and three measurements fix its ceiling. The defects that escape are mostly

absent from the text: the strongest reader identified 10 of 43, and the misses are of the environment, not of the lines. Two readers miss together ($\phi = 0.49$), so voting adds little: more judges and a vote identified 9 of 43 (Appendix B). And any function of the request text alone carries at most the information the text carries (Bound 2): checks written from the request caught 1 of 43, and checks written with the defective code in hand reproduced the defect.

Engineered readers. Open Code Review [3], Alibaba's review tool, wraps a reader in deterministic engineering: a pure file selection with a named reason per file, a sealed run identity, groups of at most ten related files reviewed in separate contexts, prose rules matched to each file kind, a six-tool contract with round and token budgets, comment positioning by a verbatim snippet matched in code, and a second model that removes a comment only when the diff proves it wrong. On AACR-Bench [2, 56], 200 pull requests from 50 repositories with 1,505 expert-verified comments scored by a judge model, it reaches precision 33.9 percent and recall 20.0 percent with Claude-4.6-Opus, against precision 15.9 and recall 12.7 for a general agent of the same model family (Claude Code with Claude-4.8-Opus), at about a fifth of the tokens (385 thousand against 2,062 thousand per pull request). The engineering raises what a reader does with the text; it does not change what the text holds (Bound 2). The gate is not scored on this benchmark: its verdict is a function of recordings, source and declared relations (Bound 1), not a comment a judge can match, and 626 of the 1,505 comments are maintainability and readability remarks no gate judges. What transfers is the shell: the named reason per excluded artifact, the sealed identity, the one-sided filter with a proof in place of the second model, and the preview a producer reads before it submits. **Equivalence.** Whether two programs compute the same function is undecidable in general [44]; verified equivalence is a theorem of a restricted language. Our proof lane translates both versions into Dafny [30] and states the subset it admits as the same program that admits a unit. Differential testing on generated inputs is the execution lane's form; its verdict is bounded to the inputs tried. **Independence.** N-version programming assumed independent failures and Knight and Leveson refuted it [27]; our dual-implementation arm reproduced the refutation (0 of 3) and our reader pairs show it as $\phi = 0.49$. **Invariants and self-written checks.** Invariants mined from data [12] are bounded by the null space of a balance (Bound 3) and caught 0 of 5. Automatically generated patches overfit the tests that accept them [31, 42]; a check the producer writes is a self-report in runnable form, which is why no check in the gate is authored by the producer. **Release.** Trust in a build rests on the toolchain [49]; diverse double-compiling [53] is one answer, a read-only image with a pinned toolchain is ours, and three independent attacks (Section 4.8) show what remains. **Artifact checks.** Workflow linters, locale-key checkers, build systems that hash declared inputs, merge queues and dangling-link hooks each decide one relation between artifacts; none decides "the repository still means the same", which Rice's theorem forbids. What transfers is the method: give each artifact a meaning, state the relation, compare parent and candidate under it, and return a proof, a counterexample or an explicit unknown (Section 3.4).

2.2 What other fields already know

We described the problem without naming software and asked, in ten research sweeps, how any field makes a wrong change to a complex system impossible, self-revealing or self-eliminating without a judge. Every citation below was opened and its claim checked against the source; the companion audit lists every source opened and every one discarded. **Accuracy from structure.** A cell copies its code with an error rate far below what one recognition step allows: an irreversible step between two chances to reject makes the rejections independent, so their error rates multiply [19, 40]; the accuracy is paid in dissipation and time [37, 43], and DNA polymerase proofreads the same way [7]. The transfer: a later gate must see evidence the earlier one lacked, with an irreversible step between them. **Independence.** Von Neumann built reliable organs from unreliable ones by majority, under independence [52]; Knight and Leveson found independently written programs fail together [27]. **Mediation.** Complete mediation checks every access by a mechanism the subject cannot bypass [46]; only the endpoints can fully verify a function [45]. The gate sits on the channel the effect travels, outside the producer's reach. **Checkpoints.** Cell-cycle checkpoints make late events depend on early ones [17]; reliability-centred maintenance inspects on a precursor signal and samples at random for what the signals miss [41]; hazard analysis places controls where a hazard can be controlled [10]. **Measurement.** Metrology states the uncertainty with the verdict [21, 24]; acceptance sampling is a sieve for a population, never a guarantee about one lot [11]; zero failures in n trials bound the rate only to about $3/n$ [16]; risk-limiting audits escalate to a full count when the sample does not settle the question [32, 48]. **Guards.** A part that fits only one way cannot be fitted wrong [50]; guards that obstruct the work are defeated [22], so the false-refusal rate is a design constraint. **Trust and incentives.** A toolchain can hide an alteration from every reader of the source [49]; a second toolchain exposes it [53]; a verifier can check a proof instead of redoing the work [14]. An unpaid verifier skips the check [34]; an optimistic system with no challenge in its first years is indistinguishable from a watch that does not work [25]; rewarding the measurable task crowds out the unmeasured one [18]. **Intent.** A reward cannot be recovered from a policy without normative assumptions [4, 38]; more than one latent explanation fits any observed distribution [33]; 30 to 38 percent of a project's requirements trace to the requester's initial ideas [1]; ambiguity detection is studied far more than removal [6]; experts reviewing one interface report mostly different problems [35, 36, 39]; people do not notice when the outcome they chose is swapped [23]. Bound 2 is the formal statement. **Conservation, codes, reversibility.** A conserved quantity that changes has found an error with certainty [5]; codes lay out redundancy by a rule with a proven bound on what they see [15, 28]; self-stabilising systems return to a legal state within a bounded span [47]; mined invariants describe the data they came from, spurious ones included [12]. Reversibility costs storage [8], a saga compensates rather than undoes [13], and removing new code worsened one of the costliest incidents on record [51]. **Judging a judge.** Two tests in two populations identify both error rates with no reference truth [20], not when their errors are dependent [26]; active testing lowers the cost

of evaluating a model [29]. Our miss rates are counts with intervals, never points from a latent model.

3 DIFFERENTIAL ATTESTATION

3.1 Overview

A *producer* (the coding agent) changes a *system* for a *requester*. A *stimulus set* Σ is a fixed list of inputs to the running system. A *recording* is everything the system does under Σ (pages, responses, rows, messages, outbound requests, resources); R is the parent's and R_c the candidate's. The *noise set* N, from two runs of the parent, lists per channel what honest runs may differ in. A *scope* S is a set of objects of the running system with the functions that implement them; an *edit* E is R restricted to S and altered by the requester into the wanted outcome. A *unit* is a function of the source; an *artifact* any other file the build reads.

Figure 1 shows the flow and Algorithm 1 the procedure. The requester selects S on real screens of the parent and writes E; a reference store seals R, S and E by hash and signature, readable and not writable by the producer. The producer writes C; before it submits, it may ask for a *preview*: the intake below, S and the shape of Σ, the subset's refusals on its draft, the predicates and sibling relations registered for the artifacts it touched, the witnesses on its files and lean's bounded verdicts on its changed units. The preview changes no state and calls the same selection function as the run, so what will be checked and what was checked cannot disagree. The gate first writes the *intake*: one row per changed file and unit with one named reason (checked, reused, refused by a named rule of the subset, failed with a class, or skipped as not TypeScript, binary, a secret path read by its name only, or beyond the run's budget), so a unit no lane looked at is a visible row. A run carries an *identity*, a hash of the two commits, the unit set, lean's commit, its library and advice tables and the verifier's version; a unit whose key (its two texts, its callees' texts, its file context, its direct imports, its configuration and the run's flags) is unchanged reuses its certificate byte for byte. The gate builds C with a fixed recipe, records it twice under Σ with every effect through a proxy, and applies the checks; each step appends to one run record that is never rewritten. On PASS a broker seals a read-only image I and mints a one-use token. In the prototypes the token is bound to a channel and a scope; binding it to $h(I)$, so that production runs I only, is the release broker's design, not yet built. Each step is a function of its inputs; the producer's description of its change is an input to none.

Algorithm 1 The gate on one candidate C.

Require: parent recordings R, R'; scope S; edit E; stimuli Σ; witness library W; predicate registry P (system layer, project layer read from the parent)

1: $T \leftarrow$ intake(C): one row and one named reason per changed file and unit ▷ the preview calls the same
2: $B \leftarrow$ build(C, recipe); **if** B fails **return** REFUSE(C2, part)
3: $R_c \leftarrow$ record(B, Σ); $R'_c \leftarrow$ record(B, Σ); $N \leftarrow \mathrm{diff}(R, R')$
4: **if** $\mathrm{diff}(R_c, R'_c) \not\subseteq N$ **return** REFUSE(C3)
5: $D \leftarrow \mathrm{diff}(R_c, R) \setminus N$; $D_{\text{out}} \leftarrow D \setminus S$; $D_{\text{in}} \leftarrow D \cap S$
6: **for all** units u of T outside S **do** $v_u \leftarrow \mathrm{lean}(u_{\text{parent}}, u_{\text{cand}})$, or the certificate stored under key(u) ▷ C4b
7: **end for**
8: **for all** artifacts a of T, predicates $p \in P(\mathrm{kind}(a))$ **do** $w_p \leftarrow p(a_{\text{parent}}, a_{\text{cand}})$ ▷ C5b
9: **end for**
10: **if** some v_u = WITNESS or some w_p = REFUSE **return** REFUSE(location, witness)
11: **if** $D_{\text{out}} \neq \emptyset$ **return** REFUSE(C4a, location)
12: **if** a relation fails in R_c (C5a) or a witness in W replays (C6) **return** REFUSE(location)
13: **if** $D_{\text{in}} \neq E$ within N **return** ASK(E, R_c)
14: **return** PASS; the broker seals I and mints a one-use token (designed: token($h(I)$))

3.2 The six checks

C1 Construction. Parts generated from one source (sibling files, locale files, derived views, provider clients, pipeline configuration) are regenerated; a hand edit to a generated part cannot exist in C. **C2 Load.** The build succeeds and the system starts in the production shape; an unresolved or unstartable part is refused by name. **C3 Reproducibility.** R_c equals R'_c within N; a recording that varies is refused as unfit for comparison. **C4 Equality.** *C4a, recordings:* D_{out} is empty and D_{in} equals E within N; a non-empty D_{out} is refused at its location, a D_{in} that differs from E returns ASK. *C4b, source:* every unit outside S is proved equal to the parent's or found equal on a stated bound of inputs; a separating input is a refusal with the input; an undecided unit is recorded and left to C4a. **C5 Relations.** *C5a, recordings:* a fact the system states in two places (a list and a detail view, a total and its rows, two locales) agrees in R_c. *C5b, artifacts:* every declared relation between artifacts holds for the candidate as it held for the parent (Section 3.4). **C6 Witnesses.** Every past escape is stored as the stimulus and the effect that revealed it, and replays against the candidate; before judging, the gate confirms it still refuses a planted set. **The rendering channel.** For a page, C3 and C4a are a comparator over pixels, under the same rule and with no reader. Its stimulus is a cell: a page, a state, a viewport, a colour scheme, the fonts the page loads and a fixed clock, rendered by a headless browser that refuses every request outside the built site. The states come from the source lane: for each function the page renders, the inputs lean's execution lane drew for it are the states the page is rendered at (the drawn arguments reach the page before its scripts run), so every state the lane reached is rendered, not a screen chosen by hand. N for the channel is built from two or more renders of the parent in the same cell: per tile of the image and per colour channel, the largest spread the parent's own renders show, extended to the box of the element that drew it; anti-aliasing, font hinting, sub-pixel layout and animation phase enter N only by appearing there. A pixel of R_c differs when a channel exceeds its bound, N times a factor the caller sets per channel: a factor of one keeps N, a stricter or a looser bound is a parameter, and where N is empty the bound stays empty under any factor. A difference between the candidate's own renders is refused by C3; a differing region outside S and outside the masked regions is REFUSE(ℓ), with ℓ the page, state, viewport, colour scheme and bounding box; inside S, R_c is compared with E, and a difference is ASK(E, R_c) with the two renders shown. The caller sets in one

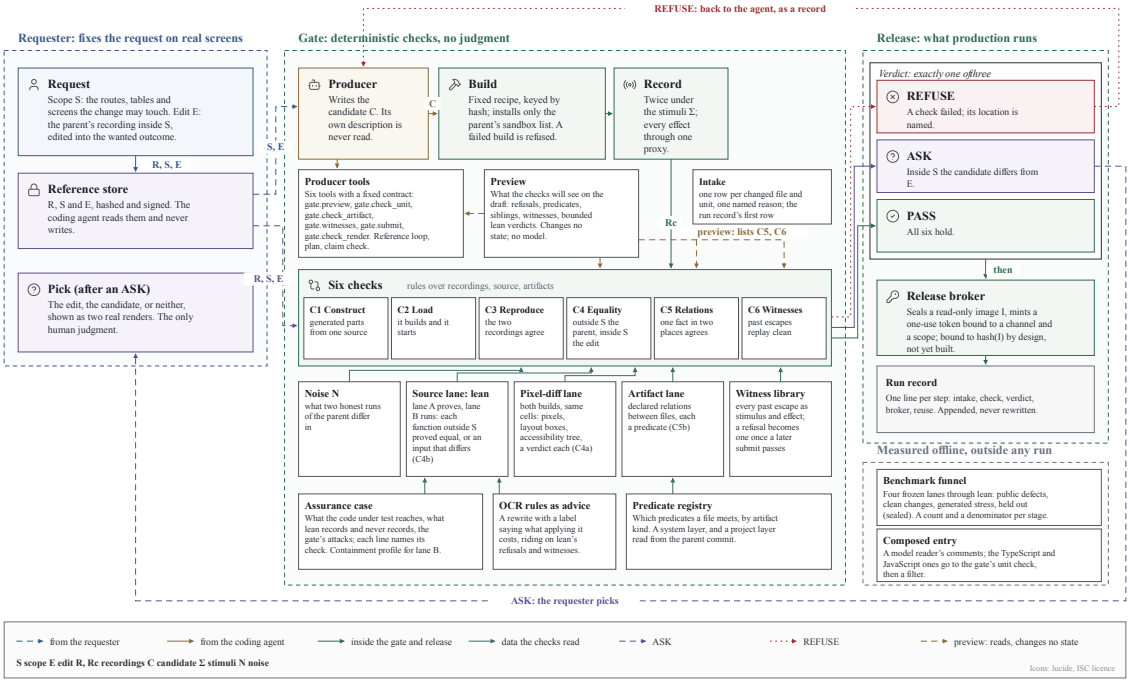


Figure 1: One change, from request to release. The requester edits a real screen into the wanted outcome; the agent builds, and before the build may read a preview that runs part of one check and lists two others; the gate writes the intake, one row and one named reason per changed file and unit, records the new build, compares it with the old one, compares the rendered cells of both builds, proves the untouched functions equal, checks the declared relations between artifacts, and applies fixed rules. The result is a release with a one-use key, a pick by the requester, or a refusal with its location; every step is appended to one run record.

versioned parameter file the viewports, the colour schemes, the tolerance, the masked regions (a clock, an avatar) and the states to render.

The verdict is PASS when all checks hold; REFUSE(ℓ) when a check failed at location ℓ, a unit and an input for C4b, an artifact and a witness for C5b; ASK(E, R_c) when C4a found a difference inside S that is not E, in which case the requester is shown E and R_c as two real renders and selects one, or neither. The selection is the only human judgment in the procedure.

3.3 The source lane

lean is a differential equivalence checker: two versions of one unit in, one of five verdicts out (Figure 2). WITNESS carries an input, the two observations that differ, the source lines on each side of the first differing effect or of the return, and the rule set's advised rewrite when it has one at that site. PASS-proved carries a theorem, checked by a verifier [30], that the versions return the same value and make the same world transitions for every input. PASS-bounded carries the inputs tried, the seed, the fakes and a time limit; it is never a proof. ASK names why the unit could not be decided. The *proof lane* translates both versions into the verifier's language: values become mathematical values, every outside call a transition of an explicit world threaded through the function, every unmodelled block an opaque transition named by its text, so a changed block is never proved equal; a theorem states that the translations agree on every input and world. The *execution lane* loads both versions in a sandbox where every outside dependency is a recording fake with seeded answers, draws inputs from the parameter types, boundary values first, and compares result, mutations and trace.

The proof lane decides only inside a *subset* of the language in which equivalence is a theorem: functions over values, bounded loops, every effect an explicit call through a port, no mutation of what the caller handed in, no module state written at the top level (Appendix C). Outside it the proof lane returns ASK with the construct named and the execution lane still runs. The subset ships as an editor rule set, the same program that admits a unit, so code written for the gate is inside it from the first line; its reach on older code is a measured number (Section 4.4).

Runs and advice. On a pull request lean writes the intake row of every changed unit: on its fixture of one unit per class, 6 rows with 6 distinct reasons, where 3 had no row before. A run may carry a time budget and a unit budget. It spends in a fixed order (changed units, then unchanged callers of changed callees), finishes the unit it is running, names its stop (time budget, unit budget or none) with the units reached and the total, and marks every unit not reached as skipped by budget; a stopped run is never green. Each rewrite in the rule set's advice table carries a label: proved,

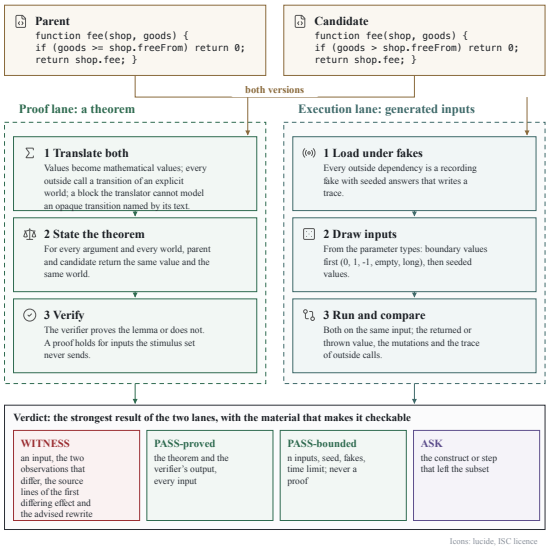


Figure 2: How lean decides one function. Left: the two versions and the scope. Middle: the proof lane translates both into the verifier's language, states that they agree on every argument and every world, and asks the verifier. Right: the execution lane loads both under recording fakes, draws the same inputs for both, and compares the result, the mutations and the trace. Bottom: the verdict with the material that makes it checkable; a WITNESS also names the source lines of the first differing effect and the advised rewrite.

conditional with its precondition, or migration. The mechanical items of Open Code Review's TypeScript checklist were sorted into it; a nested conditional expanded into nested `if` statements is proved, by a pair whose correct rewrite is PASS-proved and whose planted wrong one is a WITNESS. Entries labelled proved went from 1 to 2 of 74, beside 3 whose label is decided at the site; no refusal was added, and the subset's share of two census roots is unchanged.

3.4 The artifact lane

Ten of the 43 design-set defects are not functions: a job on a runner too small for it; a cache key that omits a changed input; a bootloaded file missing from the asset list; an immutable cache header on a rotated file; a partial locale; a view that replaces a shop's own price; a glossary reverted from a stale base; a committed absolute symlink; a credential copied without its refresh token; a singleton subscribing in its own initialiser. The artifact lane is the source lane's contract over artifacts: every artifact kind gets a meaning, every obligation between artifacts is a predicate, candidate and parent are compared under each, and the answer is a proof, a counterexample or ASK. No universal decider exists (Bound 4), so the lane is a registry of predicates.

A predicate is *derived* from an authoritative artifact with a mechanical witness (a cache key from the inputs its job reads, a translation's keys from the source locale, an asset list from the program's loads, a document's entries from the branch base), *declared* for a norm the files cannot derive and versioned outside the producer's change (a runner's memory floor, a credential's required fields, an invariant over a view), or *mined* from history, as a suggestion only. Every verdict carries its evidence class (formal, structural, empirical). A candidate is judged on what it *introduces*: a violation the parent already carried is reported, not refused. Where the files cannot decide, the lane runs the smallest thing that closes the unknown once, in an isolated environment, and records a certificate; for a view it loads both schema versions into a scratch database with a declared fixture and runs the declared invariants.

The registry is keyed by artifact kind: 26 entries map a path pattern, or a git mode for a symbolic link, to a kind and its predicates. It is a system layer under a project layer the repository versions at `.gate/project.json`, read from the parent commit, so a candidate cannot switch a predicate off. Three predicates declare sibling relations: a key added to one locale exists in every sibling locale, a key a reader reads exists in the configuration the build ships, and a declared signature matches its implementation; each returns a proof, a counterexample naming the pair, or ASK. Each of the 14 registered predicates (11 before) has a fixture pair, a parent it passes and a candidate it does not, and a predicate without one fails the registry's test. On the design set the registry maps the 427 artifacts of the repairs to the kinds they had before, and the lane still refuses 11 defects for the right reason and 2 clean changes; the sibling relations add no catch there, since every locale defect of the design set lies in the layout the older locale predicate reads.

3.5 The producer's loop

The producer is outside the gate: nothing it says enters a verdict, and the gate stays the only judge of what it submits. Five tools give it the gate's instruments under a fixed contract, over the Model Context Protocol and as command-line subcommands: the preview; lean on one unit of the draft, returning the verdict, the input, the source lines and the advice; the artifact predicates on one path; the witnesses that touch a set of files; and the submission, which runs the whole gate and returns the run record's identifier and the verdict. Every input and output is a versioned schema, a changed schema without a new version fails the contract test, and no tool calls a model. A reference loop drives the tools with no model at all: from each planted mutation of the soundness battery it calls lean on the draft, reads the WITNESS's lines, restores the parent's statement there and calls again, and it stops by name at five rounds, at a unit with no witness, or at its time budget. Before it edits, a producer writes its plan in the gate's words: each risk point with the check that sees it (C4b for a function, C5b for an artifact and its predicate, C1 for a sibling, C6 for a witness); a risk no check sees is a relation to declare in the project layer or a stimulus to ask the requester for. Every claim the producer records about behaviour is run through lean before it submits: a WITNESS refutes the claim with its input, PASS-proved confirms it, PASS-bounded keeps it with its bound and ASK leaves it open. The loop, not the gate, refuses to submit a behavioural claim nobody checked, and the check is one-sided: it refutes or confirms what the producer claimed and never supplies a claim. Every refusal is a record (the check, the location, the input or the artifact, the lines, the advice) stored under the run's identity; a pull-request comment is posted

only where no existing comment covers the same range at an intersection over union of 0.6 or more, in batches of at most 50. When a refused candidate is later submitted fixed and passes, the refusing stimulus and its effect become a witness in W, replayed by C6 on every later change to those files; before judging, the gate confirms that the stored refused candidate still fails, and aborts by name when it does not.

3.6 Four bounds

Three statements bound what any gate can observe from a recording; a fourth bounds what a proof can decide.

Bound 1 (Identical recordings). *Let g be a gate whose inputs are a recording and material not written by the producer. Two candidates with the same recording receive the same verdict, and two recordings that differ only within N cannot be separated beyond what N allows.*

Bound 2 (Rewording adds nothing). *Let I be what the requester meant and r the request text. For any function f of r alone, the information $f(r)$ carries about I is at most that of r (the data-processing inequality on $I \to r \to f(r)$; Blackwell gives the decision form [9]).*

Bound 3 (A balance has a null space). *A check that refuses when $A(o - d) \neq 0$, for an observed effect vector o, a declared vector d and a linear map A, passes every difference in the kernel of A.*

Bound 4 (Equivalence is decidable only inside a subset). *No procedure decides, for every pair of programs of a general-purpose language, whether they compute the same function [44]. For a decidable subset L, a procedure may return, for every pair in L, a proof of equivalence or a separating input; outside L it may only return a witness it found or no verdict.*

The first bound says a second reader of the same recording adds nothing: the levers are the recording, what is allowed to exist, and the source read by a procedure whose verdict is a theorem. The second says only the requester can add information about intent, by an act not determined by the text. The third says correlated checks do not add. The fourth fixes the source lane: a stated subset, a PASS outside it that is a count of inputs. A finite stimulus set certifies only its stimuli and a proof certifies every input only inside the subset, so the instruments cover each other's gap; zero refusals of n clean changes bound the false-refusal rate only to about $3/n$ [16].

4 RESULTS

4.1 Experimental setup

Corpus. We mined six repositories we operate for merged changes a later change repaired, in the manner of bug databases built from fix commits [54]. A defect enters only when the repair names it, with the repair's identifier and the verifying sentence, paired with a clean change of similar size: 43 and 43 in the design set, 101 and 101 held out, sealed by hash and opened once per design family. A second rater, blind to our labels, agreed on 7 of 10 defect classes ($\kappa = 0.61$). Each version is built once with a fixed recipe and recorded once; the parent twice, and its 1,674 self-differences on ten defects define N. **Counting rule.** A gate catches a defect only for the *right reason*: its refusal lands on what the repair addressed, read after the last counted run; every catch is re-run in a fresh tree. Every count carries a 95 percent Wilson interval [55]; zero carries the rule of three [16]. Lines are hashed before every run; a gate's builder never writes its test cases. All 72 runs, failures included, are rows of the register (Appendix A). **Baselines.** Eight other approaches ran on the same changes (Table 1); a defect an approach did not test counts as missed for it.

4.2 Approaches on the same defects

Read identified 10 of 43 defects for the right reason and refused 22 of 43 clean changes. Spec without the code wrote properties no sandbox observes, with the code reproduced the defect. Canary detected only the four boot crashes. Provider sees nothing that was not sent. Invariant is bounded by the null space of a balance. Mock reproduces the request's omissions. Challenge drew a challenge on 14 of 29 clean changes. Dual repeated the inherited error. Differential Attestation identified 40 of 43 and refused 6 of 43 clean changes. Paired, the recording checks alone identified 16 that Read missed and Read 4 that they missed (McNemar exact $p = 0.012$; with the pick 18 against 4, $p = 0.004$); the four name the failure in their change text. On clean changes 4 against 22 gives Fisher exact $p = 4 \times 10^{-5}$; each recording refusal was a defect already in the parent.

4.3 What each instrument adds

Table 2 is the ablation. The recording checks identify 22; C2 reaches the construction classes (a component omitted from the build, a dependency not visible to the loader, notation the build misreads: 6 of 11), and C3 to C5a reach derived views (2 of 5), siblings (2 of 4) and 5 of the 10 defects in the eleventh class, which holds 83 of the 144 defects in the full corpus. The source lane adds 9 defects on which a function the defect commit changed has a separating input. The requester's pick adds 2 of the 7 defects that reached ASK, with 2 of 7 clean changes refused, both visibly changed. The artifact lane adds 7: the runner, both cache keys, the symlink, the glossary write-back, the credential copy and the initialiser subscription. The 19 defects not run through C3 to C5 count as missed, so the recording count is a lower bound.

4.4 The source lane

Table 3 measures the lane on four populations, the corpus rows at lean commit c9847d8, the rest at d8f4131. *Untouched functions:* 145 top-level functions in the changed files the defect commit did not touch; the correct verdict is equal. The lane proved 39 and bounded 93; 12 were undecided, each with its reason in the register; one WITNESS showed a changed callee, a true difference. *Planted mutations:* of 143 type-checked mutations, the execution lane separated 120 and the proof lane proved none of them equal, the false-PASS count that matters; the 23 without a witness count as missed. *The corpus:* a WITNESS on a function the repair later touched is a catch: 15 of 43 on the design set (CI 22–50), 18 of 101 held out (CI 12–26); 34 of 43 and 61 of 101 had a function to run. *Reach:* the rule set admits 20,908 of 39,775 functions (53 percent) in 36 repositories we operate, written before it existed, and 39 percent of 8,921 in three public ones; the rest name the constructs the subset must

Approach	Signal read	Producer-independent	Population	Caught	95% CI	Clean refused	95% CI
Read	change text	no	43 / 43	10 / 43	13–38	22 / 43	37–65
Spec	request text, code	no	43 / 43	1 / 43	0–12	–	
Canary	runtime signals	yes	26 / 43[a]	4 / 43	3–22	–	
Provider	provider reply	yes	5 / –	1 / 5	4–62	2 / 5[b]	7–70
Invariant	parent data	yes	5 / 10	0 / 5	0–43	4 / 10	17–69
Mock	request, screenshot	no	7 / –	0 / 7	0–35	–	
Challenge	committed recording	no	20 / 29	5 / 20	11–47	14 / 29	31–66
Dual	effects of two builds	partly	3 / –	0 / 3	0–56	–	
Differential Attestation	recordings, source, artifacts, edit	yes	43 / 43	**40 / 43**	81–98	6 / 43	7–27

Table 1: Nine approaches on the same design set (Section 4.2). Population: defects / clean changes the approach was run on; a defect not run counts as missed. Caught: identified for the right reason. [a]17 defects were not runnable in the sandbox. [b]Fixed versions refused because the provider's rules had changed since the corpus was built. –: not run on clean changes. The Differential Attestation count is the union of its instruments (Table 2); its stamp is the lean commit c9847d8 (corpus rows) and the artifact-lane commit 390578a.

Instrument (cumulative)	Caught	Adds	Clean refused
Recording checks C1 to C6a	22	22	4 / 43
+ source lane, C4b (lean)	31	9	0 more
+ requester's pick on two renders	33	2	2 / 7
+ artifact lane, C5b	40	7	2 / 43
+ rendering channel			fixture only
Reached by no instrument	3 of 43: a new file loaded late, a request body no stimulus sends, a parser helper		

Table 2: What each instrument adds on the 43 design-set defects, in the order the gate applies them. "Adds" counts defects no earlier instrument caught. The recording count is a lower bound: 19 defects were not run through C3 to C5. The pick ran on the 7 defects that reached ASK. The rendering channel makes the pixel part of C3 and C4a a comparator; it has run on its own fixture only. The artifact lane's row holds under the registry keyed by artifact kind: its 14 predicates, each with a fixture pair, refuse the same 11 defects and 2 clean changes as the 11 before them.

Measurement	*n*	Result
Untouched functions, parent against defect commit	145	132 PASS
Planted mutations, parent against mutant	143	120 WITNESS
Design-set defects, right reason (c9847d8)	43	15 WITNESS
Held-out defects, right reason (c9847d8)	101	18 WITNESS
Functions of 36 repositories inside the subset	39,775	53%
Functions of n8n, pipedream, langchainjs	8,921	39%
Functions of the module written inside the subset	15	15

Table 3: The source lane; corpus rows at lean commit c9847d8, the rest at d8f4131. Untouched functions are expected equal: a PASS is the correct verdict (39 proved, 93 bounded, 12 undecided, 1 WITNESS through a changed callee). Planted mutations are expected different: 0 of the 120 witnessed were proved equal. Reach is the share of functions the rule set admits in code written before it existed; the nexus monorepo alone is 60 percent.

grow to hold. A module written inside the subset enters whole: of one change touching four functions, the lane proved one equal, bounded another, and named the slip in a third with the input that exposes it.

4.5 The artifact lane

Eleven predicates (Appendix D) ran on every defect and fix commit through the repositories' history alone; the stamp is artifact-lane commit 390578a, corpus commit 7a4ca8a. On the design set they refuse 11 of 43 for the right reason: a runner too small for its job, two cache keys without a changed input, a missing build asset, two partial locales, a credential copied without its refresh token, a stale-base write-back of a glossary, an absolute symlink, a view that replaces a shop's own price, and a singleton subscribing in its own initialiser. One defect returns ASK (a cache header in a hosting file the lane cannot yet parse); three refusals land on a file the repair did not change. On the 43 clean changes the lane refuses 2, both intended write-backs of older text the predicate names correctly; on the 72 held-out clean changes with a known repair, 4 of 72. On the 101 held-out defects it catches 1: the predicates came from the design set, and the held-out defects name the relations the registry must grow to hold. The view rung ran: both schema versions in a scratch database with a declared fixture, one row violating the declared invariant on the candidate, none on the fix.

4.6 The producer's loop

The tools' contract test passes, and it fails when a tool answers a verdict outside the enum or when a schema changes without a new version. The reference loop runs over the 143 planted mutations of the soundness battery, 120 of which lane B witnessed; its target is PASS within five rounds on every witnessed mutation and a named stop on the others. Measured at 2026-10-28 01:30 CET: not yet run to completion; the rounds to PASS and the stops are reported when the run ends (register row "The producer's loop"). The comparison of a coding agent with and without the tools on corpus defects is designed and not run.

4.7 A public corpus

The gate is not scored on AACR-Bench (Section 2): its verdicts are not comments. What follows is what lean and the artifact lane alone, with no model and no recording, cover of the benchmark's verified comments (Table 4). Of its 42 TypeScript and JavaScript

Measurement	*n*	Result
Verified comments whose named defect a refusal exhibits	413	8
partly: same function, mechanism shown, defect not named	413	+26 (34)
Refusing rows where no verified comment exists	164	100
the change the pull request asked for	100	84
not asked for	100	7
undecidable from the title and body	100	9

Table 4: lean and the artifact lane, no model, on 41 of the 42 TypeScript and JavaScript pull requests of AACR-Bench, read by one reader (an author). The gate is not scored on this benchmark; the rows say what lean alone covers. Refusals where no verified comment exists are read against each pull request's title and body as its declared scope.

pull requests, 41 were run: one head commit is not on its host, and on one change of 221 files lean ran out of memory, which leaves the artifact lane alone there. Each change ran from the merge base of the dataset's two commits, because the dataset's source commit is the target branch's tip, not the change's parent. One reader, an author, read every verified comment against the refusing rows; no judge model was used, so the counts are not on the benchmark's scale. Of 413 verified comments, a refusal exhibits the named defect for 8; with the 26 partly caught (the same function, the mechanism shown, the defect not named) it is 34. Of the rest, 223 lie in files where nothing was refused, most of them JavaScript without declared types, which lean skips; no changed function was proved equal. Of the 164 refusing rows, 100 land where no verified comment exists. Read against each pull request's title and body as its declared scope, 84 of them are the change the pull request asked for; 7 are not (three load refusals of a module the unbuilt checkout lacks, two differences the check itself made in a stack trace's file names, one unannounced change of behaviour, one undeclared environment variable), and 9 cannot be decided from the body. Without a declared scope every intended change is a refusal; with one, it is the edit C4a compares against.

4.8 Mediation, release, intent

(1) The effect proxy with a one-use token stopped 20 of 20 planted effects and 10 of 10 routes around it; a kernel sandbox alone let 12 of 20 through. (2) The first release broker refused 10 of 10 alterations planted by its builder. An independent attacker then passed 8 of 10; a second passed 8 of 10 against the repaired version; a third passed 6 of 10 against a version that seals the build into a read-only image with a pinned toolchain and refuses all 30 earlier alterations. Every successful alteration used a resource the gate shared with the producer. (3) For 20 defects whose correction was a matter of intent, the correction's content existed in the organisation's artifacts before the request in 7 cases and nowhere in 11; a fixed grammar over the request wording produced it in 1 of 20, as Bound 2 predicts. The request text available here is the producer's description, written after the build; sessions with requesters are designed and not yet run.

5 LIMITATIONS

The corpus holds only defects repaired with a note; silent defects are absent, and boot failures are over-represented in the design set. The repositories are private; the numbers, the register and the hashes are published. One team designed and judged the experiments; no outsider has re-run one. The sandbox has no network, so provider-dependent defects leave no trace. On code written before the rule set most of a change falls outside the subset and the lane returns no verdict, so the recording checks carry the gate. The proof lane models every outside call as an opaque transition, so a reordering of two independent effects is a difference it reports. A PASS-bounded is a count of inputs. The artifact lane's predicates were written from the design set and catch 1 of 101 held-out defects; a declared predicate must exist before the change it judges, and the view invariant of Section 4.5 was recorded by the repair, so that catch holds under the declared-predicate assumption only. The rendering channel's noise set bounds only what the parent's own renders showed: a visual change whose pixels stay within N (inside a region where honest renders already differ, by no larger a spread) is invisible to it, and a noise source that did not vary between the parent's renders is read as a difference. A requester who writes a wrong edit releases a wrong outcome; in the corpus 11 of 20 corrections of intent did not exist at request time.

A reader benchmark cannot measure a gate. It scores a comment that a judge matches to a verified comment; a gate's refusal names an input, a predicate or a recording instead, and on AACR-Bench a refusal exhibited the named defect for 8 of 413 verified comments and showed the mechanism without naming the defect for 26 more, while the benchmark's judge matches the concern a comment names, not the mechanism a refusal shows. The benchmark declares no scope, so a refusal of the change the pull request asked for counts against the gate: 84 of the 100 refusals where no verified comment exists were that change. And 181 of the 413 verified comments are maintainability and readability remarks that no gate judges. Our public numbers cover two of the benchmark's ten languages, rest on one reader with no second rater, and count as uncovered every comment on JavaScript without declared types, which lean skips.

6 CONCLUSION

A gate that reads neither the change nor its producer's account, and compares recordings, proves functions equal and checks declared relations between artifacts, identified 40 of 43 escaped defects against a model reader's 10, and refused 6 clean changes against the reader's 22, on the same changes under a pre-registered counting rule. Four bounds say why a second reader of the same evidence cannot do better and where the remaining defects live: in inputs no stimulus sends, in constructs outside the decidable subset, in relations nobody declared, and in intent that only the requester can supply. The target is every design-set defect, then above 90 percent of the 144-defect corpus without fitting to it, and a reach of the source lane on new code that no longer needs to be measured because the rule set keeps it at one.

REFERENCES

[1] 2022. How do requirements evolve during elicitation? An empirical study combining interviews and app store analysis. *Requirements Engineering* (2022). authors not recorded by the citation check; cited for the 30 to 38 percent traceability figure, which the check confirmed.

[2] Alibaba Group. 2026. AACR-Bench: benchmark repository and evaluation pipeline. https://github.com/alibaba/aacr-bench; dataset https://huggingface.co/datasets/Alibaba-Aone/aacr-bench. README, evaluation/README.md and docs/metrics.md read 2026-10-27; the scoring uses a judge model per file.

[3] Alibaba Group. 2026. Open Code Review: an AI-powered code review CLI. https://github.com/alibaba/open-code-review, commit 375f64c. Apache-2.0; the leaderboard figures are the project's README of 2026-10-20.

[4] Stuart Armstrong and Sören Mindermann. 2018. Occam's razor is insufficient to infer the preferences of irrational agents. In *Advances in Neural Information Processing Systems 31*.

[5] John C. Baez. 2018. Getting to the bottom of Noether's theorem. arXiv:2006.14741 (lecture notes dated 6 October 2018).

[6] Muneera Bano. 2015. Addressing the challenges of requirements ambiguity: a review of empirical literature. In *IEEE Fifth International Workshop on Empirical Requirements Engineering (EmpiRE)*.

[7] Anna Bebenek and Izabela Ziuzia-Graczyk. 2018. Fidelity of DNA replication: a matter of proofreading. *Current Genetics* 64 (2018), 985–996.

[8] C. H. Bennett. 1973. Logical reversibility of computation. *IBM Journal of Research and Development* 17, 6 (1973), 525–532.

[9] David Blackwell. 1953. Equivalent comparisons of experiments. Annals of Mathematical Statistics 24(2):265–272.

[10] Codex Alimentarius Commission. 1997. Hazard analysis and critical control point (HACCP) system and guidelines for its application. Annex to CAC/RCP 1-1969, Rev. 3.

[11] J. H. Curtiss. 1947. Acceptance sampling by variables, with special reference to the case in which quality is measured by average or dispersion. *J. Res. Nat. Bur. Standards* 39 (1947), 271–290.

[12] Michael D. Ernst, Jake Cockrell, William G. Griswold, and David Notkin. 2001. Dynamically discovering likely program invariants to support program evolution. *IEEE Transactions on Software Engineering* 27, 2 (2001), 99–123.

[13] Hector Garcia-Molina and Kenneth Salem. 1987. Sagas. In *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*. 249–259.

[14] Shafi Goldwasser, Yael Tauman Kalai, and Guy N. Rothblum. 2008. Delegating computation: interactive proofs for muggles. In *Proceedings of the 40th Annual ACM Symposium on Theory of Computing*. 113–122.

[15] R. W. Hamming. 1950. Error detecting and error correcting codes. *Bell System Technical Journal* 29, 2 (1950), 147–160.

[16] James A. Hanley and Abby Lippman-Hand. 1983. If nothing goes wrong, is everything all right? Interpreting zero numerators. *JAMA* 249, 13 (1983), 1743–1745.

[17] Leland H. Hartwell and Ted A. Weinert. 1989. Checkpoints: controls that ensure the order of cell cycle events. *Science* 246, 4930 (1989), 629–634.

[18] Bengt Holmström and Paul Milgrom. 1991. Multitask principal-agent analyses: incentive contracts, asset ownership, and job design. *Journal of Law, Economics, and Organization* 7 (1991), 24–52.

[19] J. J. Hopfield. 1974. Kinetic proofreading: a new mechanism for reducing errors in biosynthetic processes requiring high specificity. *Proceedings of the National Academy of Sciences* 71, 10 (1974), 4135–4139.

[20] Siu L. Hui and Steven D. Walter. 1980. Estimating the error rates of diagnostic tests. *Biometrics* 36, 1 (1980), 167–171.

[21] ILAC. 2019. ILAC G8:09/2019 Guidelines on decision rules and statements of conformity.

[22] Institut für Arbeitsschutz (IFA). 2015. The defeating of protective devices on machines. Focus on IFA's work No. 0252, edition 2/2015.

[23] Petter Johansson, Lars Hall, Sverker Sikström, and Andreas Olsson. 2005. Failure to detect mismatches between intention and outcome in a simple decision task. *Science* 310, 5745 (2005), 116–119.

[24] Joint Committee for Guides in Metrology. 2012. Evaluation of measurement data: the role of measurement uncertainty in conformity assessment. JCGM 106:2012.

[25] Harry Kalodner, Steven Goldfeder, Xiaoqi Chen, S. Matthew Weinberg, and Edward W. Felten. 2018. Arbitrum: scalable, private smart contracts. In *27th USENIX Security Symposium*. 1353–1370.

[26] Suzanne H. Keddie, Oliver Baerenbold, Ruth H. Keogh, and John Bradley. 2023. Estimating sensitivity and specificity of diagnostic tests using latent class models that account for conditional dependence between tests: a simulation study. *BMC Medical Research Methodology* 23 (2023).

[27] John C. Knight and Nancy G. Leveson. 1986. An experimental evaluation of the assumption of independence in multiversion programming. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 96–109.

[28] Philip Koopman and Tridib Chakravarty. 2004. Cyclic redundancy code (CRC) polynomial selection for embedded networks. In *International Conference on Dependable Systems and Networks (DSN)*.

[29] Jannik Kossen, Sebastian Farquhar, Yarin Gal, and Tom Rainforth. 2021. Active testing: sample-efficient model evaluation. In *Proceedings of the 38th International Conference on Machine Learning*.

[30] K. Rustan M. Leino. 2010. Dafny: An Automatic Program Verifier for Functional Correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16) (Lecture Notes in Computer Science, Vol. 6355)*. Springer, 348–370.

[31] Ziyu Li, Hao Yu, and Yuxin Yuan. 2026. Escaping the self-repair trap: improving test oracle generation via dual-context awareness. arXiv:2608.05917.

[32] Mark Lindeman and Philip B. Stark. 2012. A gentle introduction to risk-limiting audits. *IEEE Security and Privacy* 10, 5 (2012), 42–49.

[33] Francesco Locatello, Stefan Bauer, Mario Lucic, Gunnar Rätsch, Sylvain Gelly, Bernhard Schölkopf, and Olivier Bachem. 2019. Challenging common assumptions in the unsupervised learning of disentangled representations. In *Proceedings of the 36th International Conference on Machine Learning*.

[34] Loi Luu, Jason Teutsch, Raghav Kulkarni, and Prateek Saxena. 2015. Demystifying incentives in the consensus computer. In *Proceedings of the 22nd ACM SIGSAC Conference on Computer and Communications Security*. 706–719.

[35] Rolf Molich and Joseph S. Dumas. 2008. Comparative usability evaluation (CUE-4). *Behaviour and Information Technology* 27, 3 (2008), 263–281.

[36] Rolf Molich, Meghan R. Ede, Klaus Kaasgaard, and Barbara Karyukin. 2004. Comparative usability evaluation. *Behaviour and Information Technology* 23, 1 (2004), 65–74.

[37] Arvind Murugan, David A. Huse, and Stanislas Leibler. 2012. Speed, dissipation, and error in kinetic proofreading. *Proceedings of the National Academy of Sciences* 109, 30 (2012), 12034–12039.

[38] Andrew Y. Ng, Daishi Harada, and Stuart Russell. 1999. Policy invariance under reward transformations: theory and application to reward shaping. In *Proceedings of the 16th International Conference on Machine Learning*. 278–287.

[39] Jakob Nielsen and Rolf Molich. 1990. Heuristic evaluation of user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '90)*. 249–256.

[40] Jacques Ninio. 1975. Kinetic amplification of enzyme discrimination. *Biochimie* 57, 5 (1975), 587–595.

[41] F. Stanley Nowlan and Howard F. Heap. 1978. *Reliability-Centered Maintenance*. United Airlines, for the Office of the Assistant Secretary of Defense.

[42] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. 2015. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 2015 International Symposium on Software Testing and Analysis*. 24–36.

[43] Riccardo Rao and Luca Peliti. 2015. Thermodynamics of accuracy in kinetic proofreading: dissipation and efficiency trade-offs. *Journal of Statistical Mechanics: Theory and Experiment* (2015). arXiv:1504.02494.

[44] H. G. Rice. 1953. Classes of recursively enumerable sets and their decision problems. *Trans. Amer. Math. Soc.* 74, 2 (1953), 358–366.

[45] J. H. Saltzer, D. P. Reed, and D. D. Clark. 1984. End-to-end arguments in system design. *ACM Transactions on Computer Systems* 2, 4 (1984), 277–288.

[46] Jerome H. Saltzer and Michael D. Schroeder. 1975. The protection of information in computer systems. *Proc. IEEE* 63, 9 (1975), 1278–1308.

[47] Marco Schneider. 1993. Self-stabilization. *Comput. Surveys* 25, 1 (1993), 45–67.

[48] Philip B. Stark. 2008. Conservative statistical post-election audits. *Annals of Applied Statistics* 2, 2 (2008), 550–581. https://doi.org/10.1214/08-AOAS161

[49] Ken Thompson. 1984. Reflections on trusting trust. *Commun. ACM* 27, 8 (1984), 761–763.

[50] Iris D. Tommelein. 2008. Poka yoke or quality by mistake proofing design and construction systems. In *Proceedings of the 16th Annual Conference of the International Group for Lean Construction*.

[51] U.S. Securities and Exchange Commission. 2013. SEC charges Knight Capital with violations of market access rule. Press release 2013-222.

[52] John von Neumann. 1956. Probabilistic logics and the synthesis of reliable organisms from unreliable components. *Automata Studies (Annals of Mathematics Studies 34)* (1956), 43–98. lectures of 1952.

[53] David A. Wheeler. 2009. *Fully countering trusting trust through diverse double-compiling*. Ph. D. Dissertation. George Mason University.

[54] Ratnadira Widyasari et al. 2020. BugsInPy: a database of existing bugs in Python programs to enable controlled testing and debugging studies. Proceedings of ESEC/FSE 2020, tool demonstration; repository commit 11c5f1ee used.

[55] Edwin B. Wilson. 1927. Probable inference, the law of succession, and statistical inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212.

[56] Lei Zhang, Yongda Yu, Minghui Yu, Xinxin Guo, Zhengqi Zhuang, Guoping Rong, Dong Shao, Haifeng Shen, Hongyu Kuang, Zhengfeng Li, Boge Wang, Guoan Zhang, Bangyu Xiang, and Xiaobin Xu. 2026. AACR-Bench: Evaluating Automatic Code Review with Holistic Repository-Level Context. *arXiv preprint arXiv:2601.19494* (2026).

A THE EXPERIMENT REGISTER

Every experiment, including the killed ones. The register file holds eleven columns per row (date, question, design and population, pre-registration file and hash, lines, result, verdict, files, spend, caveats); the three shown here are the question, the result and the verdict. Rows marked RUNNING or PENDING are reported as they stood on 26 October 2026.

Table 5: The experiment register: every run, its pre-registered lines and its verdict. Dates are 2026. Counts are N of M as recorded in the register at the time of writing; a RUNNING row is reported as it stood.

Question	Result (N of M)	Verdict
Do text judges find real escaped bugs?	Claude flags 18/43 bugs, 11/43 clean; Codex 26/43, 25/43; the real bug named by either on 9/43	judges near chance without a frozen intent
Are two judges independent?	phi 0.49 Claude-Claude; 0.40 to 0.44 Opus-Sonnet; a second family adds 2 catches	not independent (Law 1)
Is one judge repeatable?	Claude flips 20 percent (kappa 0.53), Codex 7 percent (0.86); 5 runs voted = 1 run	too noisy to gate on
Does CI colour predict escapes?	defect rate 57 percent with green rollup, 33 percent with red	CI does not predict; red-block withdrawn
Can two judges' overlap estimate unseen bugs?	estimate 29 vs true 43	killed: dependent channels
Do planted bugs measure the miss rate?	decoys missed 20 percent (13 kind-matched); real missed 77 and 62 percent	killed as a proxy (Law 4)
Does request-vs-final-code comparison catch intent drift?	8/43 detected, 2/43 false alarms, none the real bug	killed
Does an effects ledger catch effect discrepancies?	7/7 classes detected 5/5, 0 false alarms	kept; blind to wrong logic
Do checks written from the request alone catch the bug?	2/43 after one repair round (1 in round 1); 76/300 checks fail on fixed	killed
With the buggy code in hand?	1/43, not reproduced (jest worker crash) = 0/43	killed
Outcomes first, then checks in code?	2/43 valid at final tree names, 1 MATCH (pyka-71971fb); 157/328 checks fail on fixed	killed
Syndrome gate pre-check	7/43 fire on buggy (6 run on trees); upper bound 15/43 with rules written from the escape	weak
First-article gate: do bugs concentrate in first use of a new thing?	fit 8/43 vs 2/43 (4.0, p 0.089); held-out 9/100 vs 6/100 (1.5, CI 0.55 to 4.06, p 0.59); size ranking catches 8 to 10	inconclusive
Provocation battery	44 percent in sample; 3 to 19 percent held out	weak
Do sentinel journey KINDS transfer?	Codex 55/101, Opus 22/101, both 21/101; kappa 0.34	KILL
Two-signal anergy pre-check	v1 anergic 448/903 (49.6 percent), v2 fail 127/903 (14.1 percent)	pre-check only
Does a 10-minute canary window show damage, with and without traffic?	26/43 runnable; idle 4/26, driven 4/26 (same four boot crashes, 5 to 26 s); 0/22 among apps that stayed up; 4 buggy builds fail to build (2 on the escaped bug)	KILL for traffic; boot and build probes kept (8/43)
Throughput as stake: does a quota cut on escapes lower the latent stock?	stock -42.2 percent but 396/917 held; random holds -42.5; K1 p 0.0685; audit alone -1.4 percent at read rate 9/43	INCONCLUSIVE (no gain over random)
Do two model families disagree on effects where one is wrong?	0/3 right-reason (1 partial); 754: shared inherited header; 3304d63: stimuli never reached the view; 1192: boot class only	NO-GO
Do environment-issued receipts refuse construction bugs?	5/5 and 0/5, same with public rules only; median 159 s	PASS (pilot)
Blind full replay of settlement: do the receipts (resolve, load, reserve, couple, read-back) refuse the buggy builds of all 43 bugs and spare the 43 clean and 43 fixed, with exercisers written from the change alone?	classes 6 of 10 (1,2,3,4,5,7; 2 and 3 without the six known bugs); buggy refused 13 of 43, confirmed catches 10 of 43, 5 of 37 without known bugs, all on the load leg; clean 4 of 43 (three inherited defects, one likely false alarm), none introduced by the change; fixed 2 of 43 (one non-discriminating, one wrong-reason real break); 7 desktop and nutshell bugs not final	INCONCLUSIVE
Does a paid challenger with a committed trace find real escapes?	at 12:19: 27 challenges arbitrated, 9 valid proofs, 5 right step	pending
Does the previous build as the spec, balanced against claims, show the uncovered bugs?	1 bug through the harness	pending
Can a judge name the odd replay of three?	owner 4/7 bugs (each phrase judged right by the lead; 753 and 235 are pixel-identical in the replay, 950b9d1 missed), 2/7 clean called (both visibly changed); chance tail 0.17; panel 7/7 bugs (2 guesses), 3/7 clean; parent-vs-parent noise 0 of 12 shots	INCONCLUSIVE (4 of the 5 visibly different bugs called)
Do invariants mined from the old build's data break on the new?	bugs 0/5; clean violating 4/10 (timestamp-uniqueness and background-job noise); mined sets 89 to 768 per change; fix text first read after all units	KILL
Do invariants mined from a production-shaped seed on the parent flag the buggy build and spare clean changes?	bugs 3 of 5 (687, 3304d63, a724c3d); clean 1 of 10 (684, a periodic job's rate band)	INCONCLUSIVE
Does the real provider sandbox refuse the artifact's exact request?	right reason 1/5 (754); any reason 2/5; accepted 2/5 (928, 1995); indeterminate 1/5 (930); fixed refused 2/5 because the corpus's fixed shas for 754 and 762 are themselves broken at today's provider	KILL
Does a picture approved before the build separate buggy from fixed?	0/7: 3112 pick closer to buggy, 950b9d1 within noise, 235 identical on screen, 753 state not reproduced, 2808/800/3186 owner picked none (no acceptable mock in 3 of 7)	KILL
Are our bug labels one person's opinion?	archetype 7/10 agree (kappa 0.61); symptom class 6/7 (kappa 0.82); 3 archetype disagreements where the rater chose none	labels hold; disagreements listed
Can the canary window be reproduced from a package?	A: 26 runnable, idle 4/26, driven 4/26, same four boot-crash keys; B: 24/24 fields on 1321 and 754; 28/28 on 1130 and 785 (all four boot-crash signals confirmed under the hashed harness; 1192 28/28)	PASS (same author; the independent re-run is a separate row)
Collect ten blind deep-research briefs (a research report to a research report) on Parallel ultra through one Nexus workflow; each returns mechanisms, ten cheap experiments, null results, theory, tooling, ranking, sources	10 of 10 completed, 0 failed, 100 candidate experiments, 355 source URLs (250 in Parallel's basis, 105 not)	PASS (collection only; content UNVERIFIED)

Question	Result (N of M)	Verdict
Screen: does co-change history name the sibling a buggy change left out?	0 of 8 (6 no recommendation, 2186 a wrong file)	SCREEN-KILL
Screen: do declared fact relations between sibling artifacts fail in the buggy tree only?	2 of 4 (2808 locale keys; a724c3d two ingest_run definitions, new)	SCREEN-PASS
Screen: does a stale producer fingerprint in a dependent artifact mark the buggy tree?	2 of 4 carry provenance, none resolves to a producer	SCREEN-INCONCLUSIVE (not evaluable)
Screen: does a judge reading execution traces, citing a line and allowed to abstain, name the real defect?	0 of 10 (7 claims, none the labelled defect; 5 cite failures the parent or fixed build shows too); control 0 of 10	SCREEN-KILL
Screen: does a region-level pixel diff after a stability gate localise the UI bug?	2 of 10 (753 desktop 2-tile region; 3112 region spans the page); 6 cannot win by construction (no fixed PNG)	SCREEN-INCONCLUSIVE
Screen: do rules mined from the other corpus2 fixes fire on a held-out buggy diff?	0 of 10 (0 of 7 evaluable)	SCREEN-KILL
Screen: does a stock static ruleset, reported only on new lines, land on the defect?	0 of 10: 13 new findings on 8 buggy changes, none at the defect	SCREEN-KILL
Screen: do cross-file checks on workflow, gate and config files flag the buggy version only?	0 of 4; one finding on 1850 also fires on fixed	SCREEN-KILL
Screen: does the emitted request shape, read from source, break the provider contract in the buggy version only?	0 of 6: no new finding; 754 and 762 findings identical at parent; Notion, Langfuse, prompt text have no contract	SCREEN-KILL
Screen: does literal import and reference resolution find a broken reference the path-existence control misses?	0 of 8 by the frozen rule: 119 has 12 real broken named exports but on a different break from the escaped lazy-load (and the the receipts load receipt already refuses 119); 1172 has 11 false positives	SCREEN-KILL (adjudicated: a true positive on a different break is not the escaped bug)
Confirmation (blind, held out): do the sibling fact relations frozen from corpus2 fail in the buggy tree of the held-out sibling bugs?	wins 0 of 10 (parent arm quiet; positive control 2808 reproduces); clean 1 of 10 (PR 4704, four locales missing 2 keys)	KILL
Public replication: are text judges near chance on real bugs outside our corpus?	right reason 11 of 40 = 28 percent [16, 43]; flagged bugs 24 of 40, clean 15 of 40, gap 22.5 (run 1); run 2 gap 25.0 on the line; on the 23 true introducers 10 of 23	PASS (replicates)
Public replication: are planted bugs easier to catch than real ones?	planted missed 0 of 33 [0, 10]; real missed 16 of 40 = 40 percent [26, 55]; gap 40 points	PASS (replicates)
Public replication: do settlement receipts refuse construction bugs on a public corpus?	2 of 81 refused (1 right reason; one a build timeout at load 223 to 265), clean 0 of 20; positive controls 5 of 6 refused	KILL (fails to replicate)
Prospective shadow arm: do the gates, run in observe mode on every driftbench merge for 14 days, refuse the merges later fixed as bugs and spare the rest?	PENDING: observer pid 38250 started 23:41:16Z; dry run 3 of 3 WOULD_PASS; positive controls 3 of 3 refused	PENDING
Census: how many of the 144 escapes could not have been written if siblings, mirrored types, locale files, fixtures, derived views, provider clients and pipeline config were generated from one source?	IMPOSSIBLE 23 (8 firm, 15 marginal), POSSIBLE 120, UNCLEAR 1; sibling 9 of 18, derived view 1 of 10, none-of-ten 7 of 83	CENSUS: claim not met (23 of 144, about 16 percent)
Screen: does parent-only structural matching (names, paths, types, neighbourhoods) name the omitted sibling?	0 of 4 (2808 ranks fr first but clean and fixed also raise proposals; 453 and a724c3d cannot win under the rule); control 0 of 4	SCREEN-KILL
Instrument screen: does a dependence-aware latent-class interval contain the simulated miss rate where the independence model does not?	dependence-aware 6 of 10; independence 3 of 10	SCREEN-KILL (as an instrument: neither model is trustworthy near zero)
Instrument screen: does the Allman-Matias-Rhodes identifiability check classify identifiable vs non-identifiable latent models correctly before fitting?	9 of 10 by the scored rule (one spec unresolved by EM at its cap; post hoc identifiable, 10 of 10); naive control 4 of 10, strict control 9 of 10	SCREEN-INCONCLUSIVE
Instrument screen: does the identified set of the miss rate return the full admissible range under an unrestricted latent-label construction?	10 of 10 (range [0,1] on 8 tables; labelled miss rate inside the tightest assumption level on all 10)	SCREEN-PASS
Census: how many of the 33 machinery bugs altered the admission machinery, and would the identity check (gate, reference, comparator, release key outside the producer's reach) have refused them?	ALTERED-GATE 18, OTHER 4, NOT 10, UNCLEAR 1; the identity check refuses 16 of 18 by ownership, 0 by identity, 2 not	CENSUS
Does a served build bound by hash to what was checked, with a signed reference and comparator the producer cannot write, catch planted alterations?	10 of 10 caught, 5 of 5 controls accepted; naive hash and the bench manifest test 5 of 10 each; two further variants missed by v1, fixed in v2 (11 of 11)	SCREEN-PASS
Census: would a requester-fixed footprint, enforced at the permission level, have denied the defective change?	4 of 34 DENIABLE (range 2 to 11 on 9 borderline calls); 20 of 34 fixes a strict subset of the defective footprint	CENSUS: line not met
Census: can each corpus2 request's scope be expressed as a selection of existing identifiers of the parent?	26 of 43 EXPRESSIBLE; all 17 failures are requests about a new thing; 39 of 43 counting attachment points	CENSUS: line not met
Census: what share of the 144 fixes is a sequence of twenty constructors with preconditions?	24 of 144 YES, 116 NO, 4 unclear; 14 of 20 constructors never used	CENSUS: line not met
Does a proxy on every effect channel plus a one-use token gate stop effects without a token and routes around the proxy?	10 of 10 stopped (9 blocked, 1 held) and 10 of 10 blocked; enforcement off 20 of 20 effects produced; kernel profile alone 12 of 20, tokens alone 5, both 3; the bench's profile alone lets 10 of 20 escape	SCREEN-PASS
Blind attack: can a lane that did not build the identity check, version 2 identity check and never saw its alteration list make it accept an untested build?	2 of 10 caught; 8 missed (forged bytecode, sibling shadow module, symlinked root, unlistable directory, check-to-use window, snapshot rename, PATH shim, self-minted check folder); controls accepted	SCREEN-KILL
Replay equality inside a named scope, stage 1 on the parent comparison's recordings: is every difference outside the identifiers the request names a refusal that lands on the defect, and are clean changes spared?	wins 5 of 21 (refused elsewhere 10, accepted 1, scope empty 6); clean refused 4 of 8; all 29 verdicts identical with an empty scope	INCONCLUSIVE
Closure re-run: does instrument v3 (the eight fixes from the blind attack) catch both prior alteration sets?	20 of 20 caught, 7 of 7 controls accepted, 11 of 11 probes caught	PASS (closure, not blind)
Census: does a fixed grammar of presupposition and purpose triggers over the request text list the correction among at most ten candidates?	1 yes, 2 partly, 17 no	CENSUS: line not met
Census: was the correction's content knowable at request time in the org's own artifacts, or created afterwards?	knowable 7 (35 percent), created later 11, unresolved 2; direct-only 30 percent, upper bound 45	CENSUS
Census: does an earlier correction by the same requester carry the same omission class as the later bug?	16 yes (14 strong, 2 weak), 4 no; control person 8 of 20; same area within 14 days 4 of 20 (post hoc)	CENSUS: line met by volume, not by prediction

Question	Result (N of M)	Verdict
Second blind attack on instrument v3 by a fresh lane that saw neither alteration list	2 of 10 caught; 8 missed (interpreter startup hook, bytecode in the prefix folder, node wrapper, template database contents, rewrite after the last re-hash, world code at record time, minted anchor and check folder, PATH of the starting process); extra: the stub spawns before the re-check	SCREEN-KILL
Paid proof: the producer submits a runnable proof of its own change; a second family challenges; validity re-run twice on the change and twice on the parent	right step 5 of 20 (5 of 13 buildable; 5 of 18 reclassified); false alarms 14 of 29 (13 net); attention 4 of 5; Codex 9 of 20 vs Claude 5 of 29; phi vs the second implementation 0.086	INCONCLUSIVE
Census: of the corrections knowable at request time, how many are machine-checkable, and does a mechanical lookup by the request's identifiers retrieve them?	2 machine-checkable, 5 prose; 3 of 7 with an existing check that fails on the defect (per fix commits, not re-run); recall 6 of 7; median 309 artifacts per bug, 25 relevant of 22,196 (99.9 percent noise); strict filter: recall 3 of 7	CENSUS: line met, mechanism unusable (noise)
Closure re-run: does instrument v4 (a read-only image, a pinned toolchain, a sandboxed check with the tested identity as an input, a signed template digest) catch all three prior alteration sets?	30 of 30 caught (21 refused by the gate, 3 denied by the OS, 6 left the served build intact); 9 of 9 controls accepted; boot from the image healthz 200 in 0.11 s	PASS (closure, not blind)
Screen: do conservation equations over the database snapshots and the effects ledger break on the defective recording and hold on the parent?	0 of 6 broken at the defect, 0 elsewhere; null 0 of 13; control 0 of 6; a planted break fires in four families	SCREEN-KILL (partial)
Screen: does a change that truncates or extends an object kind's lifecycle, mined from the parent's recordings, get refused at the defect?	at the defect 1 of 6 (route reading), 0 of 6 (kind reading); refused anywhere 2 of 6; null 0 of 15	SCREEN-INCONCLUSIVE (partial; KILL under the strict reading)
Screen: does set-valued acceptance (the candidate must lie inside the set the parent's two recordings span, per channel) refuse every bug and accept the parent's own second run?	null accepted 6 of 15; bugs refused 15 of 15, at the defect 3 of 15; fixed refused 15 of 15	SCREEN-KILL (partial)
Census: can each intent correction be expressed as an edit of the parent's recording such that the defective recording differs from the edit and the fixed recording equals it within noise?	STRICT 0 of 6, LENIENT 0 of 6: check B fails on every bug; two fixed sides render error pages on 90 and 86 of 108 route-viewports (capture defect, cause not isolated)	CENSUS (partial): line not met
Third blind attack on instrument v4 (read-only image, pinned toolchain, sandboxed check, signed template digest) by a fresh lane shown none of the three alteration lists	4 of 10 caught (older signed set replay, world flip during record, backing-file flip, hostile environment); 6 missed (same-user signing key, serve-on use-time replay, a volume stacked on the mount point, template clone written after its digest, same-user toolchain tree, port squat)	SCREEN-KILL
Screen: do published interface norms, encoded as mechanical checks over recorded state and applied to the object kind the request names, catch the intent bugs?	0 of 5 caught, 0 of 5 testable; control 0 norm failures on 30 parent captures	SCREEN-KILL (partial)
Does a source-level equivalence checker (lean: a proof lane and a bounded-execution lane over the parent and the defect commit) catch the escaped bug for the right reason?	design 11/43 (CI 15 to 40), held-out 18/101 (12 to 26); DA+lean 27/43, with the pick 29/43; untouched functions 132/145 PASS (1 WITNESS through a changed callee); planted mutants 120/143 witnessed, 0 false PASS; reach 60.0 percent of 48,696 units	PASS at lean d8f4131; superseded by the c9847d8 run
Same question, lean at c9847d8 (class readings, dynamic reads, callbacks, spread, React handlers; three Swift defects ported to TypeScript)	design 15/43 (CI 22 to 50), held-out 18/101 (12 to 26); DA+lean 31/43, with the pick 33/43; with the artifact lane 40/43	PASS
Published reader benchmark (not re-run by us): what does AACR-Bench report for an engineered reader and a general agent?	AACR-Bench: 200 pull requests, 50 repositories, 1,505 verified comments (626 maintainability and readability), scored per file by a judge model. Open Code Review with Claude-4.6-Opus: precision 33.9 percent (301 of 889), recall 20.0 percent (301 of 1,505), 385 thousand tokens per pull request; Claude Code v2.1.169 with Claude-4.8-Opus: precision 15.9 percent (191 of 1,200), recall 12.7 percent, 2,062 thousand tokens. Tokens per pull request: about a fifth for Open Code Review (385 against 2,062 thousand); an earlier version of Section 2 said a ninth, which was wrong. Source: lean-v3/research/open-code-review/dossier-v2.md 7.1 and 7.3 (arXiv 2601.19494 v3; the leaderboard image in OCR's README at 375f64c)	SOURCED (published; a reader benchmark, not a gate measurement)
Phase A pieces in lean (intake rows, run identity, witness lines, run budgets, advice entries), each on lean's own fixtures	intake: 6 rows with 6 coverage values on the fixture of one unit per class, 3 of 6 with no row before (red: secret-path rule off, 5 rows); run identity: a second run reused 4 of 4 units, certificates byte-identical, wall 19.8 percent of the first (one run, loaded machine; red: callee texts dropped from the key, 2 callers stay reused); witness lines: 10 of 10 sampled pairs carry a candidate line that holds the construct the pair names (red: the unit's first line, 10 of 10 fail); budgets: –time-budget 1 stops at 0 of 3 units, –unit-budget 2 at 2 of 3, a stopped run exits 1 under –block ask (red: stop removed, exit 0); advice: entries labelled proved 1 of 71 before, 2 of 74 after, 3 decided at the site; census share unchanged on two roots (87.9 and 71.6 percent). Lane commits 5720a9d, ac6aeb5, 4b945ff, 18b4399, fa06d03; reports lean-v3/harness/lanes/intake-rows.md, run-identity.md, witness-lines.md, budgets.md, advice-rules.md	PASS on fixtures; corpus measures not yet run
Registry keyed by artifact kind, with sibling relations as declared predicates: does every predicate refuse its fixture candidate and pass its parent, and does the lane keep its share on the design set?	26 registry entries; 14 predicates, each with a fixture pair (11 before), 14 of 14 green; red runs: swapped pair, missing pair, renamed kind, merge mutant, each exit 1; 43 repairs, 427 artifacts mapped to the same kinds; design set caught for the right reason 11 before and 11 after, clean refused 2 and 2; the three sibling predicates add no catch; locale-siblings refuses nexus-2808 (a sibling locale lacks 60 keys the change added) and passes its repair. Source artifact-lane/results-b2/test-registry.log and score.txt before and after; lean-v3/harness/lanes/gate-registry.md	PASS (share unchanged; no catch added)

Question	Result (N of M)	Verdict
The producer's loop: do the gate's tools hold their contract, and does a scripted producer reach PASS on the planted mutations by reading the WITNESS lines?	tools: 5 (preview, check_unit, check_artifact, witnesses, submit) over MCP and as CLI subcommands; contract test 10 of 10 pass; red runs (a verdict outside the enum; a schema changed without a new version) exit 1; no model client in the gate's own files. Refusal records and posting rules: lean's ci tests 8 of 8 pass, red run (overlap threshold 0) plans duplicates and exits 1 (lean 65cf919, 6cdd0a1). Reference loop: run over 143 planted mutations (soundness-v7, 120 witnessed by lane B) started 2026-10-28 01:25 CET (gate/loop/v7-143.json); measured at 01:30 CET: not yet run to completion. Plan in the gate's words, claim self-check, witness library: in progress. Source lean-v3/harness/lanes/gate-tools.md, refusals-json.md, producer-loop.md	PENDING (loop counts not yet measured)
Public corpus, no model (D1): register R7-aacr-D1, how many verified comments on AACR-Bench's TypeScript and JavaScript pull requests do lean and the artifact lane cover, and how often do they refuse where no comment exists?	42 pull requests (29 TypeScript, 13 JavaScript; 417 verified comments); 41 run: nodejs#56185 unreachable (its 4 comments out of the denominator), n8n#15362 (221 changed files) out of memory in lean at 8 and 16 GB, artifact lane only; 40 with a lean table. Run from the merge base of the dataset's two commits (its source commit is the target branch tip, not the change's parent); pr.mjs with –time-budget 600 –unit-budget 60; the artifact lane through its registry. Caught for the right reason 8 of 413 verified comments; partly caught (same function, mechanism shown, defect not named) 26 more, 34 of 413 with them; 223 in files where nothing was refused; no changed function proved equal. Refusing rows 164; 100 of them where no verified comment exists (96 WITNESS, 3 load refusals, 1 artifact). Read against the pull requests' titles and bodies as the declared scope: 84 the asked-for change, 7 not asked for, 9 undecidable. One reader for every comment and scope reading, no model. Source lean-v3/research/open-code-review/baseline/TABLE.md (sha256 713db691), GT-READING.md (7395e578), SCOPE-READING.md (d81f90bb); harness/lanes/aacr-baseline.md, aacr-scope.md; lean b1ea2ca	MEASURED (baseline; one reader; the composed entry, D3, not run)
Wording check: what does the one-use token bind in the prototypes the paper measured?	the mediation prototype's token is bound to a channel and a scope string; neither the identity-check instrument nor the mediation prototype binds it to the image hash h(I) in its code as read (lean-v3/harness/lanes/assurance.md, finding 4). Section 3.1, Algorithm 1 and Figure 1 now state the h(I) binding as the release broker's design, not yet built; the measured counts of Section 4.7 are unchanged	CORRECTED (text)

B DISCARDED DESIGNS, WITH THE REASON

One line per design that died, in the order it was tried. The reason is what the data showed, not what we expected.

Table 6: Designs discarded and why.

Design (origin)	Result	Why it failed
More judges, then vote (jury theorem)	right reason 9 of 43; ϕ 0.49	both judges read the same text; most defects are not in it
Capture-recapture (ecology)	estimate 29 against 43 true	the two samplers shared blind spots
Planted defects as calibration (proteomics)	missed 20 percent planted, 77 percent real	plants are local and visible; escapes are wrong about the world
Request against final code (back-translation)	8 of 43 flagged, none the defect	text against text; the gap was code against environment
Red pipeline as a signal	57 percent defects merged green	tests exercise what their authors imagined
Checks from text, three forms	2, 0, 2 of 43	without the code the checks miss; with it they copy the code
First-article inspection (aerospace)	ratio 4.0 on the design set, 1.5 held out	tuned on the same data; escapes are where the environment is
Provocation battery (stress testing)	44 percent in sample, 3 to 19 held out	fit to known defects
Sentinel journeys (indicator species)	raters 55 and 22 of 101, κ 0.34	journeys are defect-specific; nine kinds, no dominant one
Canary window (canary analysis)	4 of 26, all boot crashes	the driver never reached where the defects lived
Occupancy models (ecology)	0.09 separation	repeated runs of one detector are not independent surveys
Throughput as stake (proof of stake)	no gain over random holds	escapes are not concentrated by producer; labels arrive late
Dual implementation (aviation)	0 of 3	a shared mistake in the context is shared
Mined invariants, one database (Daikon)	0 of 5, 4 of 10 clean	the seed never filled the corrupted tables; a balance has a null space
Provider sandbox as the oracle	right reason 1 of 5	blind to everything the provider never receives
Picture approved before the build	0 of 7	a mock from the request is faithful to an incomplete request
Co-change history; sibling structure	0 of 8; 0 of 4	the omitted sibling is not predicted by what moved together before
Rules mined from repairs; static rules; config checks	0 of 10; 0 of 10; 0 of 4	nothing fired at the defect
Trace reader with abstention	0 of 10	the cited failures were in the parent too
Sibling fact relations, held out	0 of 10 blind	the held-out siblings have a different shape

Design (origin)	Result	Why it failed
Requester-fixed footprint	4 of 34 deniable	the repair lies inside the frame the requester would fix
Grammar of twenty constructors	24 of 144	real repairs are free edits
Wording grammar for intent	1 of 20	the wording does not contain the correction
Requester history for intent	16 of 20 against a control's 8	volume, not prediction
Lookup of organisation artifacts	recall 6 of 7 at 99.9 percent noise	unrankable
Paid challenger with forced errors	5 of 20; 14 of 29 false alarms	a proof the producer writes is a self-report in runnable form
Identity check, versions 2 and 3	2 of 10 caught, twice, blind	every miss lay outside what the files' hashes cover

C THE SOURCE LANE: SUBSET AND VERDICT CONTRACT

Verdicts. A verdict is one of five, each with the material that makes it checkable by a reader who does not trust the checker. WITNESS: the input, the two observations (returned or thrown value, mutations of the arguments, the trace of outside calls with their arguments) and the first field in which they differ. PASS-proved: the two translations, the theorem and the verifier's output; the theorem quantifies over every argument and every world. PASS-bounded: the number of inputs, the seed, the fakes used and the time limit; it carries no claim beyond the inputs tried. ASK: the construct or the step at which the unit left the subset, with its line. REFUSE: the policy violated.

The subset. A unit is inside the subset when it is a function over values, with: numbers as integers (a quotient is written as an explicit floor), strings, booleans, records with named fields, sequences, sets and maps; `let` and `const` locals, with a local changed only by assignment; conditionals, `switch` over literals, `for`–`of` and counted `for` loops, and `while` with a stated bound; early `return`; calls to same-file functions, to pure built-ins, and to the methods of sequences, strings, sets and maps; callbacks of sequence methods and local lambdas; a callback handed to an outside call as an opaque value named by its text; a type parameter, compared by value; a defaulted parameter and a destructuring default; an optional call, whose transition happens only when the receiver is present; a method call on an opaque value as a transition of the world; an effect only as a call through a value the function received (a port), each such call a transition of the world in program order; a thrown error as a fault raised on the world; classes whose methods read their fields and write them through the world. Outside the subset: mutation of what the caller handed in, module state written at the top level, `this` outside a method, reflection, dynamic keys on records without an index type, generators, unbounded loops, and function values in positions the readings do not cover. The lint names the construct and proposes the form inside the subset; the same program admits a unit to the proof lane.

Translation. Each unit becomes a function of the verifier's language. Every call through a port becomes an uninterpreted function of the world and the arguments that returns a new world and a value; the world is threaded through the unit in program order, so two versions that call the same ports in the same order on the same arguments compute the same world. A block the translator does not model becomes an uninterpreted transition named by the hash of its own text, with the locals it reads as inputs and the locals it assigns as outputs; a block whose text changed is a different transition and is never proved equal to the old one. A recursive unit takes a fuel argument. The theorem for a pair states that the parent's function and the candidate's function return the same value and the same world for every argument and every world, with every same-file callee outside the scope assumed equal by the same theorem.

D THE ARTIFACT LANE: PREDICATES RUN

Eleven predicates, each run through the repositories' history (`git show`, `git diff-tree`; nothing checked out) on the defect commit against its parent and on the fix commit against its parent. Derived: *cache-key-inputs* (a job's cache key hashes every file the job's own run steps read; a changed recipe with an unchanged key is refused), *locale-complete* (every key the change adds to the source locale exists in every target locale; pre-existing gaps are reported, not refused), *asset-listed* (every file a changed source file loads is covered by the build's asset globs), *base-preserved* (a change that writes back the older text of a document over lines a landed commit had replaced is refused with that commit named), *symlink-absolute* (no committed link points at an absolute path). Declared: *runner-floor* (a job's measured memory floor against the runner it moves to), *credential-fields* (a script that copies a credential record keeps the fields its consumer requires), *init-subscribes* (no singleton subscribes to a synchronous publisher inside its own initialiser), *cache-header-scope* (an immutable cache header applies only to names carrying a content hash), *sql-view-invariants* (declared invariants over a view, run on a scratch database loaded with both schema versions and a declared fixture). Structural, ASK: *sql-view-changed* (a view body changed and no solver reads the dialect). Every declared predicate judges only a violation the change introduces.

E DATA AND CODE AVAILABILITY

The repositories are private, and the corpus holds change text, so the corpus, the recordings and the logs are not published. The register, the pre-registration files with their hashes, the harness library, the analysis scripts, the citation-check files and this paper's source are held with the work and are available to reviewers under agreement. Every number in this paper names the file it was read from in the register. No experiment has yet been re-run by anyone outside the team; this is stated as a limitation, not a plan. The reference-by-reference audit of the 462 sources opened across the literature sweeps, with each verdict and whether this paper uses the source, is a companion document.